

Twinkle — upgrading a database that is already in use

A version is installed at a clinic. Everything after it has to **add to** that database, never replace it. A fresh installation still has to get the whole schema in one go.

Both come from the same mechanism, and it is already in place.

The short answer

EF Core migrations, which the application already applies at startup. There is no separate set of upgrade scripts to run and nothing to remember to execute in order — that is the whole point of them.

On every launch, `DbBootstrapper.InitialiseAsync` calls `Migrate()`. It:

- reads `__EFMigrationsHistory`, a table inside the clinic's own database, to see which migrations that file has already had,
- applies **only the ones it has not**, oldest first,
- writes each one into that table as it goes.

A new PC has no database, so it has no history, so it gets all of them — which is how "create the whole schema" and "add the two new columns" end up being the same line of code.

```
Opening the existing database at C:\HMS\DB\twinkle.db.  
Applying migrations: 20260729103000_AddCustomerPhone  
Backed up before upgrading to 20260729103000_AddCustomerPhone: C:\HMS\DBBackup\pre-upgrade-20260729-103014.db  
Database ready.
```

That is in `C:\HMS\Logs` after every launch, and it is the first thing to read when somebody asks whether an upgrade did anything.

Making a schema change

1. Change the entity

`src/Pharma.Core/Entities.cs`, as normal.

2. Add a migration

```
dotnet ef migrations add AddCustomerPhone --project src\Pharma.Data --startup-project src\Pharma.Data
```

`--startup-project src\Pharma.Data`, not `Pharma.App` — the WPF project does not reference `Microsoft.EntityFrameworkCore.Design`, and the tools stop with *"Your startup project 'TwinkleHMS' doesn't reference..."* if you point them at it.

Name it after what it does. That name appears in the clinic's log and in their `__EFMigrationsHistory` for the life of the database.

3. Read what it generated

`src/Pharma.Data/Migrations/<timestamp>_AddCustomerPhone.cs`. Two minutes, every time. You are looking for anything that drops or rebuilds a table when you only asked for a column — see [SQLite](#).

4. Run the tests

```
dotnet test tests\Pharma.Tests\Pharma.Tests.csproj --filter UpgradeTests
```

Five tests, two seconds. They are described below.

5. Ship it

Build the installer as usual. The migration travels inside the executable; there is nothing extra to copy and nothing to run by hand at the clinic.

What the tests behind this actually check

`tests/Pharma.Tests/UpgradeTests.cs`. Worth knowing what they cover, because before them **nothing tested upgrading at all** — every other test in the project migrates a file that does not exist yet, so migrations were only ever exercised as "create everything from nothing".

Test	What it would catch
Every change to the model has a migration behind it	An entity changed with no migration added. The column would be missing at the clinic and the application would come up fine, then fail on the screen that uses it
An old database keeps its records when it is upgraded	The clinic's actual path: a database built by the old version, with a patient in it, gaining new columns. Asserts the patient is still there afterwards and the new columns can be written to
The migrations apply one at a time in order	A migration that only works when the whole chain runs in one go. A clinic three versions behind takes them one step at a time
Upgrading twice is the same as upgrading once	Re-running the installer, or opening the application twice, being treated as a special case
A new installation is created at the latest version	A fresh PC not getting the newest migration

The first is the one that earns its keep. It is `dotnet ef migrations has-pending-model-changes`, run somewhere it cannot be forgotten.

The backup taken before an upgrade

When there are migrations to apply and a database already exists, the application copies it first:

```
C:\HMS\DBBackup\pre-upgrade-20260729-103014.db
```

These are kept for good and are not part of the daily rotation. Two reasons the ordinary daily backup was not enough:

- It takes one per day and skips if today's already exists. A clinic that opened the application before installing the new version has already spent it.
- Daily backups are pruned to the newest few. The only copy of the old shape could be deleted a week later, which is precisely when somebody asks for it.

If a migration fails, the log says so and the file is still there. Recovery is copying it back over `C:\HMS\DB\twinkle.db` with the application closed, and reinstalling the previous version.

Rules that keep this working

Never change a migration that has shipped. The clinic's database has already recorded that it ran. Editing it changes what a *new* installation gets and nothing about theirs, so the two drift apart silently. Always add another one.

Never delete a migration. Same reason, and it breaks the chain for anyone more than one version behind.

Never call `EnsureCreated()` . It builds the schema without recording any history, so the database can never be migrated afterwards. Nothing in this project uses it and nothing should start.

Never ship a `.db` file with the installer. It would overwrite the clinic's records. The installer copies one executable and nothing else, deliberately.

Bump `<Version>` in `Pharma.App.csproj` for every release. Not required by migrations, but it is how you find out which schema a clinic is on when they ring up.

What SQLite makes awkward

SQLite cannot alter a column or drop one in place. When a migration needs either, EF Core **rebuilds the whole table** — makes a new one, copies the rows across, drops the old, renames. It works, and it is worth knowing because:

- It is slow on a large table. A clinic with two years of bills will notice.
- A rebuild is where data is silently reshaped if a type changed. Read the generated migration.
- Adding a **new nullable column** is the cheap case and needs no rebuild. Prefer it where the choice exists.

Adding a `NOT NULL` column to a table with rows in it needs a default, or the migration fails at the clinic and nowhere else — there are no rows on your machine to fail against. Either give it a default or add it nullable and fill it in.

Changing data rather than schema

The seeding in `DbBootstrapper.SeedAsync` only runs when a table is **empty**. It sets up a new installation and will never touch a clinic that already has doctors and medicines.

So a data fix for existing installations — correcting a GST rate, repairing bad rows — is not seeding. Options, in order of preference:

1. **A migration with SQL in it**, using `migrationBuilder.Sql(...)` . Runs once per database, recorded in the history like any other. Right for a one-off correction.
2. **A repair the user can run and see**, like the existing **Settings** → **Check data health**. Right when the fix needs judgement or ought to be visible.

What not to do is put a data fix in the startup path with a "have I done this yet" flag of its own. `__EFMigrationsHistory` is that flag, and it already works.

If you ever need the plain SQL

To hand a DBA the script, or to see what an upgrade will do without running it:

```
dotnet ef migrations script --project src\Pharma.Data --startup-project src\Pharma.Data --idempotent --output upgrade.sql
```

`--idempotent` wraps each step in a check, so the script is safe to run against a database at any version. Useful for reading. **Not** how upgrades are delivered — the application applies its own migrations, and a hand-run script that half matches is worse than none.